AI-Powered Customer Support Automation System

Documentation Report

Course: Agentic AI

Assignment 2

Submitted By

Vansh Anand

Registration Number

23MID0105

VIT Vellore

Table of Contents

1. Project Overview

This project presents an AI-Powered Customer Support Automation System developed using **LangGraph**, **LangChain**, **Ollama (Llama 3.1)**, **SQLite**, and **Retrieval-Augmented Generation (RAG)**. The system automates customer query handling through multiple specialized agents. It classifies customer intent, routes queries to the appropriate department, retrieves relevant information from company documents, stores conversation history using SQLite, supports memory recall, incorporates Human-in-the-Loop approval for high-risk requests, and validates responses through a Supervisor Agent before presenting the final response.

Core Capabilities

Capability	Description
Intent Classification	Classifies customer queries into Sales, Technical, Billing, or Account using Llama 3.1
Conditional Routing	Routes queries to specialized support agents based on detected intent
RAG Retrieval	Retrieves relevant information from the knowledge base using ChromaDB
Specialized Agents	Dedicated Sales, Technical, Billing, and Account support agents
SQLite Memory	Stores and recalls previous customer conversations

Human Approval	Requests supervisor approval for high-risk operations (refunds, cancellations)
Supervisor Agent	Reviews, validates, and improves generated responses before delivery
Final Response	Produces the validated, professional response for the customer

2. System Architecture

The system follows a directed graph architecture implemented using LangGraph's `StateGraph`. Each node in the graph represents a processing stage, and edges define the flow of execution. The shared state (`CustomerState`) is passed through every node, accumulating information as the query progresses through the pipeline.

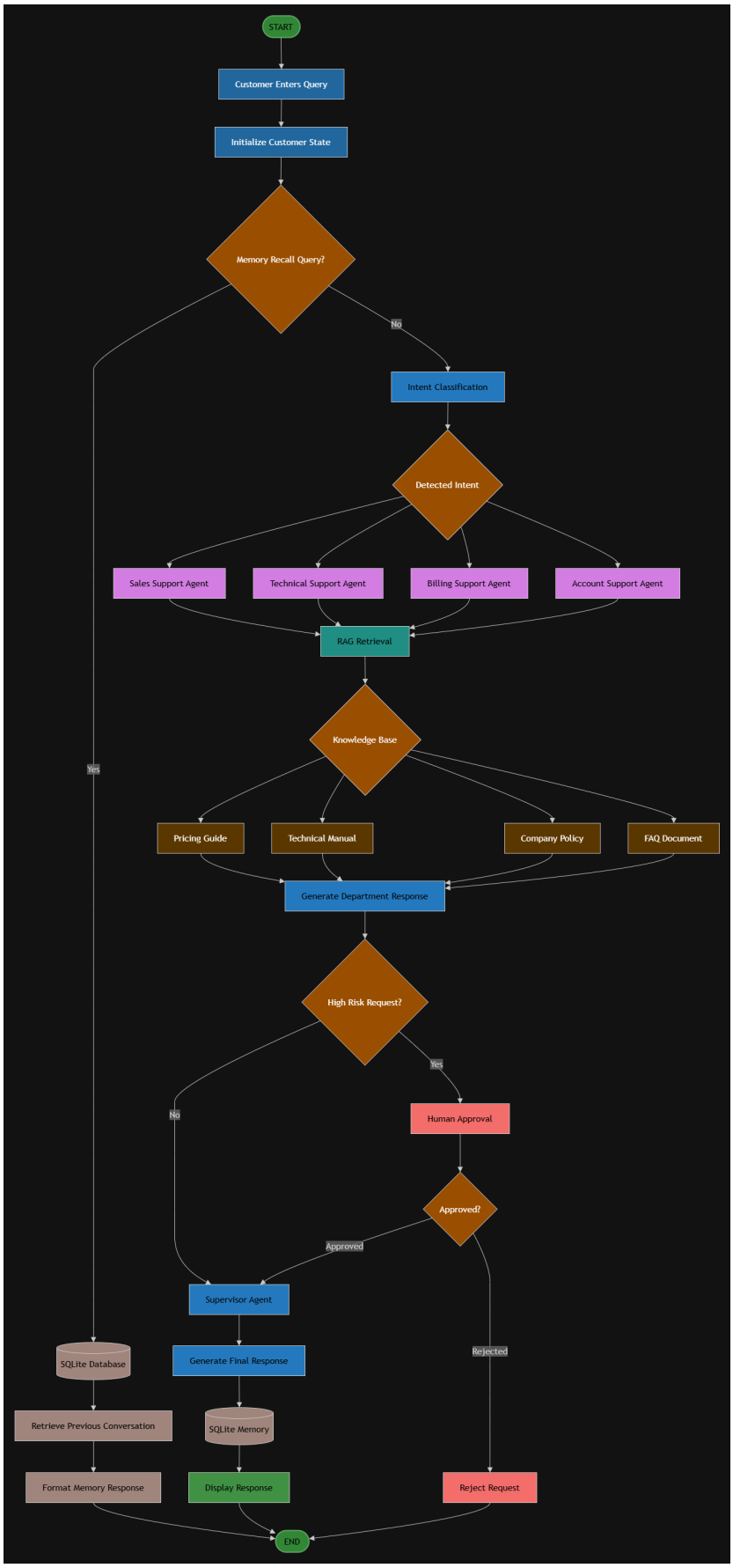


Figure 1: Overall LangGraph Workflow Architecture

Architecture Explanation

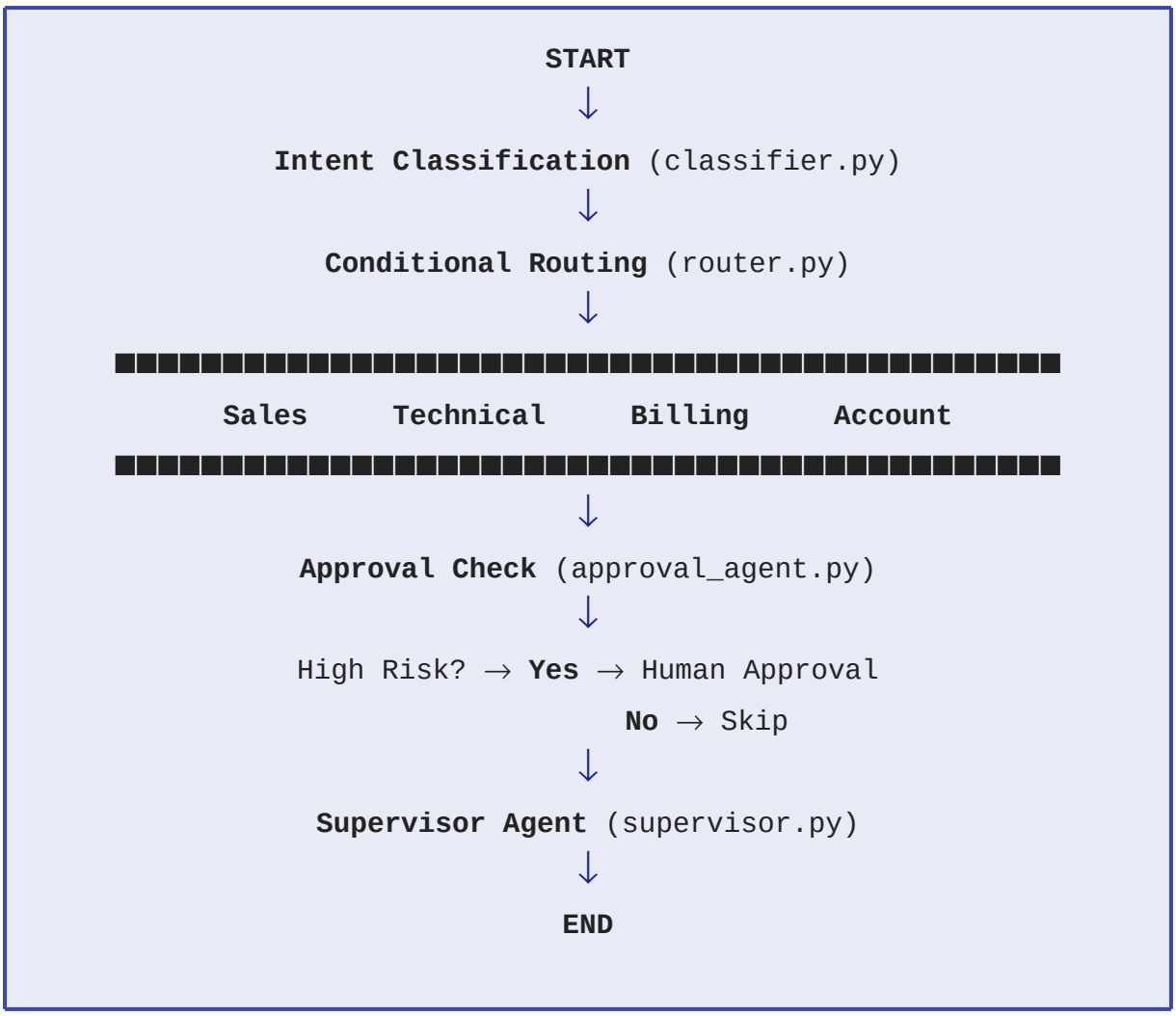
- **State Initialization:** The workflow begins by initializing a `CustomerState` dictionary containing customer name, query, and empty fields for intent, context, responses, and approval flags.
- **Memory Recall:** If the customer asks about a previous support issue, the system queries SQLite directly and bypasses the LangGraph pipeline.
- **Intent Classification:** The customer query is sent to Ollama's Llama 3.1 model, which classifies it into one of four departments: Sales, Technical, Billing, or Account.
- **Conditional Routing:** Based on the classified intent, the query is routed to the appropriate specialized agent using LangGraph's `add_conditional_edges`.
- **Specialized Agents:** Each department agent processes the query and generates a department-specific response. Sales and Technical agents also retrieve RAG context.
- **RAG Retrieval:** Relevant information is retrieved from the ChromaDB vector store containing company documents.
- **Human Approval:** The approval agent checks for high-risk keywords. If found, the workflow routes to a human approval node where manual approval is required.
- **Supervisor Agent:** The supervisor validates the response, combines RAG context with the department response, and generates the final professional output.
- **SQLite Storage:** The complete conversation (query, intent, response, timestamp) is saved to SQLite for future recall.
- **Final Response:** The validated response is displayed to the customer.

3. Technology Stack

Component	Technology
Programming Language	Python 3.13
Framework	LangGraph
AI Framework	LangChain
LLM	Ollama (Llama 3.1)
Embeddings	HuggingFace (all-MiniLM-L6-v2)
Vector Store	ChromaDB
Database	SQLite
IDE	Visual Studio Code

4. LangGraph Workflow

The LangGraph workflow is defined in `graph/workflow.py`. It connects all nodes into a single executable directed graph.



Node Descriptions

Node	Function	Description
classifier	`classify_intent()`	Uses Llama 3.1 to classify queries into Sales, Technical, Billing, or Account
sales	`sales_agent()`	Handles sales queries and retrieves pricing information via RAG

technical	`technical_agent()`	Handles technical issues and retrieves troubleshooting steps via RAG
billing	`billing_agent()`	Handles billing, invoice, and refund-related queries
account	`account_agent()`	Handles password resets and account management
approval	`approval_agent()`	Checks for high-risk keywords and flags requests requiring human approval
human	`human_approval()`	Prompts for manual approval/rejection of flagged requests
supervisor	`supervisor_agent()`	Validates response, combines RAG context, and generates final output

5. Module Description

Project Structure

```
CustomerSupportAutomation/
■■■ agents/
■   ■■■ classifier.py
■   ■■■ sales_agent.py
■   ■■■ technical_agent.py
■   ■■■ billing_agent.py
■   ■■■ account_agent.py
■   ■■■ approval_agent.py
■   ■■■ supervisor.py
■■■ graph/
■   ■■■ state.py
■   ■■■ router.py
■   ■■■ workflow.py
■■■ rag/
■   ■■■ loader.py
■   ■■■ rag_pipeline.py
■■■ memory/
■   ■■■ sqlite_memory.py
■■■ knowledge_base/
■   ■■■ company_policy.txt
■   ■■■ pricing_guide.txt
■   ■■■ technical_manual.txt
■   ■■■ faq_document.txt
■■■ database/
■   ■■■ customer_support.db
■■■ diagrams/
■   ■■■ workflowDiagram.png
■■■ app.py
■■■ README.md
■■■ requirements.txt
■■■ .gitignore
```

agents/ — Support Agent Modules

File	Description
`classifier.py`	Uses Ollama's Llama 3.1 model with zero temperature to classify the customer query into one of four intents: Sales, Technical, Billing, or Account.
`sales_agent.py`	Handles sales-related queries. Retrieves pricing information from the knowledge base using the RAG pipeline and generates a sales-focused response.
`technical_agent.py`	Handles technical issues. Retrieves troubleshooting steps from the technical manual via RAG and provides step-by-step guidance.
`billing_agent.py`	Handles invoices, payments, and refund-related queries. Flags refund requests for human approval.
`account_agent.py`	Handles password resets, profile management, and account-related operations securely.
`approval_agent.py`	Scans queries for high-risk keywords (refund, cancel, compensation, etc.) and flags them. Contains the `human_approval()` function for interactive approval.

`supervisor.py`	Final validation layer. Combines RAG context and department response into a professional final response. Handles approval rejection scenarios.

graph/ — Workflow Orchestration

File	Description
`state.py`	Defines `CustomerState` as a `TypedDict` with fields: customer_name, user_query, intent, retrieved_context, department_response, final_response, approval_required, and approval_status.
`router.py`	Contains the `route_query()` function that reads the classified intent and returns the corresponding agent node name for conditional routing.
`workflow.py`	Builds the complete `StateGraph`, registers all nodes, defines edges (including conditional edges for routing and approval), and compiles the executable graph.

rag/ — Retrieval-Augmented Generation

File	Description
`loader.py`	Loads all `.txt` files from the `knowledge_base/` directory using LangChain's `TextLoader`.
`rag_pipeline.py`	Creates a ChromaDB vector store from document chunks, uses HuggingFace embeddings (`all-MiniLM-L6-v2`), and provides the `retrieve_context()` function for similarity search.

memory/ — Conversation Persistence

File	Description
`sqlite_memory.py`	Manages SQLite database operations: `initialize_database()` creates the table, `save_conversation()` stores interactions, and `get_last_conversation()` retrieves the most recent conversation for a customer.

app.py — Main Application

The unified entry point that initializes the database, accepts customer input, checks for memory recall queries, executes the LangGraph workflow, saves the conversation to SQLite, and displays the final response.

6. RAG Pipeline

The Retrieval-Augmented Generation (RAG) pipeline ensures that the system provides accurate, fact-based responses by retrieving relevant information from company documents before generating answers.

Pipeline Components

Component	Implementation
Document Loader	`TextLoader` from LangChain Community
Text Splitter	`RecursiveCharacterTextSplitter` (chunk_size=500, overlap=50)
Embedding Model	HuggingFace `all-MiniLM-L6-v2`
Vector Store	ChromaDB (persisted to `./database/chroma_db`)
Retrieval Method	Similarity Search (k=1)

How It Works

1. **Loading:** All `.txt` files from the `knowledge_base/` folder are loaded using `TextLoader`.
2. **Chunking:** Documents are split into chunks of 500 characters with 50-character overlap using `RecursiveCharacterTextSplitter`.

3. **Embedding:** Each chunk is converted into a vector representation using the HuggingFace `all-MiniLM-L6-v2` model.
4. **Storage:** Vectors are stored in a ChromaDB vector store persisted on disk.
5. **Retrieval:** When a query arrives, the system performs a similarity search to find the most relevant chunk (k=1) and returns it as context.

Knowledge Base Documents

Document	Content
`pricing_guide.txt`	Subscription plans (Starter $10/month, Professional $30/month, Enterprise custom pricing) with features
`technical_manual.txt`	Common troubleshooting steps: restart, update, verify connectivity, check disk space, reinstall, contact support
`company_policy.txt`	Company policies and guidelines
`faq_document.txt`	Frequently asked questions and answers

7. SQLite Memory

The system uses SQLite to persist every customer interaction, enabling conversation recall across sessions.

Features

- **Conversation Storage:** Every interaction (query, intent, department response, final response) is automatically saved with a timestamp.
- **Previous Issue Recall:** When a customer asks "What was my previous support issue?", the system queries SQLite to retrieve the most recent conversation for that customer.
- **Persistent Memory:** Data is stored in `database/customer_support.db` and persists across application restarts.

Database Operations

Function	Description
`initialize_database()`	Creates the `conversation_history` table if it does not exist
`save_conversation(state)`	Inserts a new conversation record from the current state
`get_last_conversation(name)`	Retrieves the most recent conversation for a given customer name

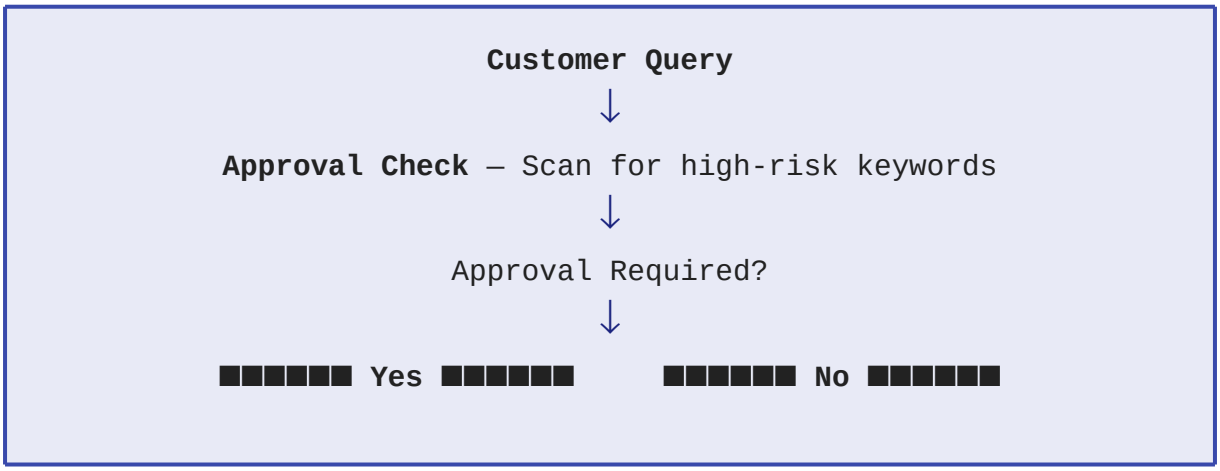
8. Human-in-the-Loop

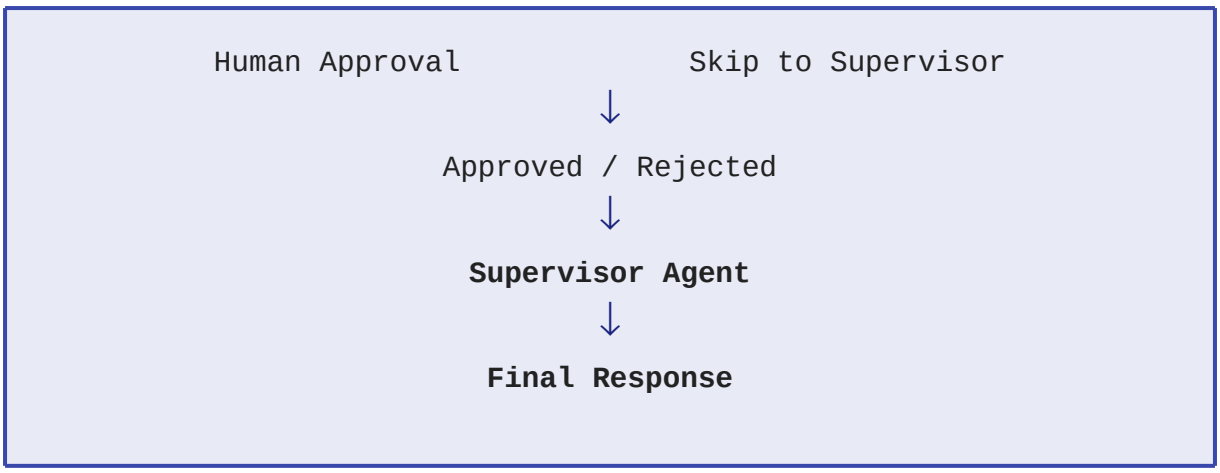
High-risk customer requests are not processed automatically. They require explicit human approval before the system proceeds.

High-Risk Keywords

Keyword	Scenario
`refund`	Refund requests
`cancel` / `cancellation`	Subscription cancellation
`account closure` / `close account`	Account closure requests
`compensation`	Compensation requests
`management`	Escalation to management

Approval Workflow





If the request is **approved**, the Supervisor Agent generates the final response normally. If the request is **rejected**, the Supervisor Agent returns a rejection message: *"Your request cannot be processed because it was not approved by the supervisor."*

9. Supervisor Agent

The Supervisor Agent is the final validation layer before any response reaches the customer.

Responsibilities

- **Response Validation:** Checks whether human approval was required and whether it was granted or rejected.
- **Context Integration:** If RAG context was retrieved, it is formatted and prepended to the response under "Relevant Information".
- **Professional Formatting:** The department response is formatted under "Support Response" with clear section separators.
- **Final Response Generation:** Appends a professional closing message: "Thank you for contacting ABC Technologies."

Output Structure

```
Relevant Information
----------------------------------------
[Retrieved RAG Context]

Support Response
----------------------------------------
[Department Agent Response]

Thank you for contacting ABC Technologies.
```

10. Task Completion

Task	Description	Status
Task 1	Project Setup and Initial Configuration	■ **Completed**
Task 2	State Structure (CustomerState TypedDict)	■ **Completed**
Task 3	Intent Classification using Llama 3.1	■ **Completed**
Task 4	Conditional Routing with LangGraph	■ **Completed**
Task 5	Specialized Support Agents	■ **Completed**
Task 6	RAG Pipeline (ChromaDB + HuggingFace)	■ **Completed**
Task 7	SQLite Conversation Memory	■ **Completed**
Task 8	Human-in-the-Loop Approval	■ **Completed**

Task 9	Supervisor Agent	■ **Completed**
Task 10	Complete System Integration and Demonstration	■ **Completed**

11. Installation

Prerequisites

- Python 3.11 or higher
- Ollama installed with Llama 3.1 model pulled
- Git

Setup Steps

Step 1: Clone the repository

```
git clone https://github.com/Vansh-Anand/AI-Powered-Customer-Support-Automation-System
cd CustomerSupportAutomation
```

Step 2: Create a virtual environment

```
python -m venv venv
venv\Scripts\activate
```

Step 3: Install dependencies

```
pip install -r requirements.txt
```

Step 4: Ensure Ollama is running

```
ollama pull llama3.1
```

Step 5: Run the application

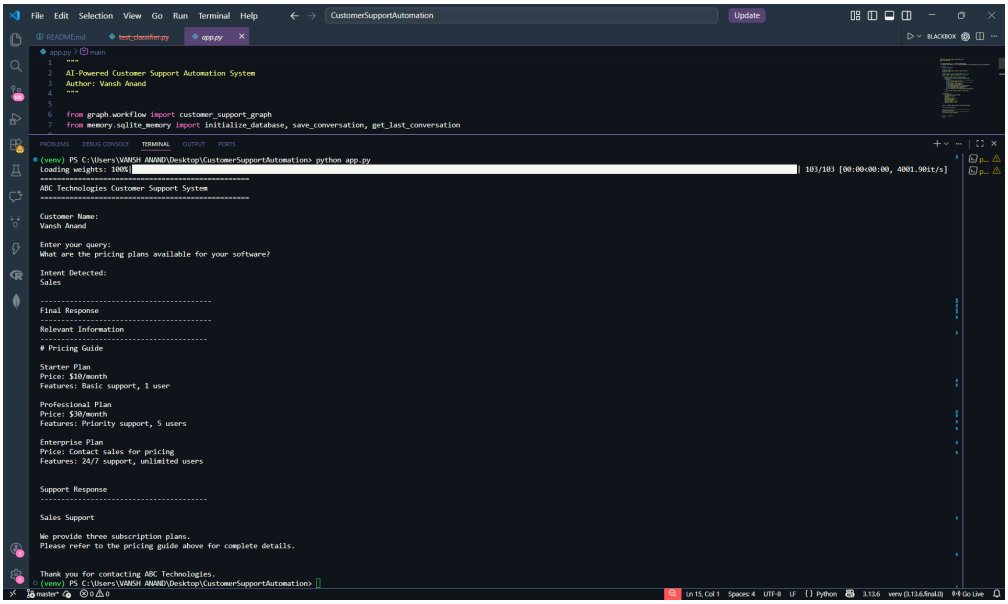
```
python app.py
```

12. Sample Execution

Query 1 — Sales (Pricing Plans)

Input: *"What are the pricing plans available for your software?"*

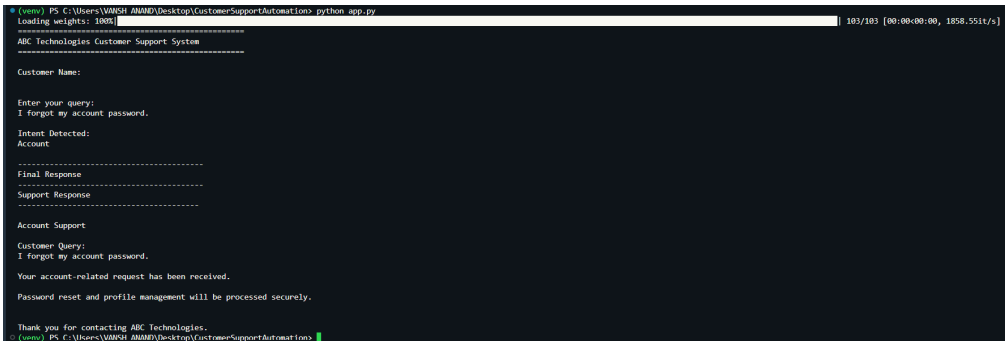
The system classifies the intent as **Sales**, retrieves pricing information from the knowledge base via RAG, and presents the Starter, Professional, and Enterprise plans to the customer.



Query 2 — Account (Password Reset)

Input: *"I forgot my account password."*

The system classifies the intent as **Account** and routes the query to the Account Agent, which provides guidance on password reset and profile management.



Query 3 — Technical (Application Crash)

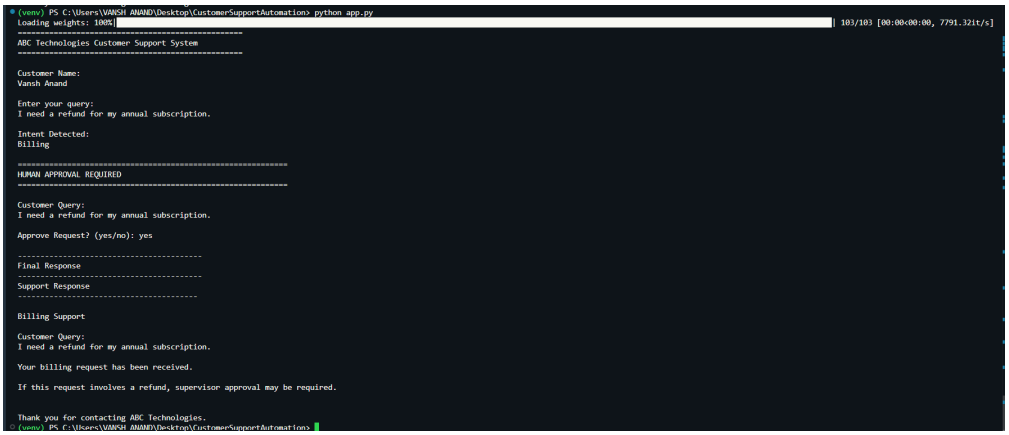
Input: *"My application crashes whenever I upload a file."*

The system classifies the intent as **Technical**, retrieves troubleshooting steps from the technical manual via RAG, and provides step-by-step resolution guidance.

Query 4 — Billing with Human Approval (Refund)

Input: *"I need a refund for my annual subscription."*

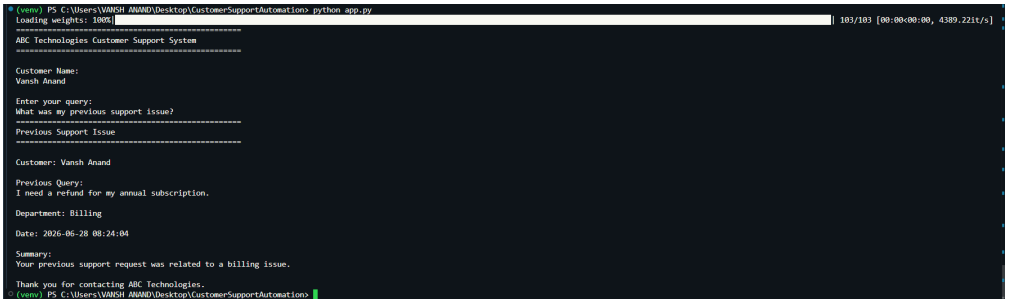
The system classifies the intent as **Billing**, detects the high-risk keyword "refund", and triggers the Human-in-the-Loop approval workflow. The supervisor processes the approved request.



Query 5 — Memory Recall (Previous Issue)

Input: *"What was my previous support issue?"*

The system queries the SQLite database for the customer's most recent interaction and displays the previous query, department, date, and a summary.



13. Knowledge Base

The knowledge base consists of four text documents stored in the `knowledge_base/` directory. These documents are loaded, chunked, embedded, and stored in the ChromaDB vector store for retrieval.

File	Description
`company_policy.txt`	Contains company policies, guidelines, and operational procedures for ABC Technologies.
`pricing_guide.txt`	Lists subscription plans with pricing and features: Starter ($10/month, 1 user), Professional ($30/month, 5 users), and Enterprise (custom pricing, unlimited users).
`technical_manual.txt`	Provides common troubleshooting steps: restart, update, connectivity check, disk space verification, reinstallation, and escalation to technical support.
`faq_document.txt`	Contains frequently asked questions and their answers for quick customer reference.

14. Database Schema

The project uses a single SQLite database file: `database/customer_support.db`

Table: conversation_history

Column	Type	Description
`id`	INTEGER PRIMARY KEY AUTOINCREMENT	Unique identifier for each conversation
`customer_name`	TEXT	Name of the customer
`user_query`	TEXT	The original customer query
`intent`	TEXT	Classified intent (Sales, Technical, Billing, Account)
`department_response`	TEXT	Response generated by the department agent
`final_response`	TEXT	Validated final response from the Supervisor Agent
`timestamp`	DATETIME	Auto-generated timestamp (DEFAULT CURRENT_TIMESTAMP)

SQL Schema

```
CREATE TABLE IF NOT EXISTS conversation_history(
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   customer_name TEXT,
   user_query TEXT,
   intent TEXT,
   department_response TEXT,
   final_response TEXT,
   timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

15. Conclusion

This project demonstrates a complete, end-to-end AI-Powered Customer Support Automation System that integrates multiple modern AI and software engineering concepts:

- **LangGraph Workflow:** A directed graph architecture that orchestrates the entire customer support pipeline through well-defined nodes and edges.
- **Specialized Support Agents:** Four dedicated agents (Sales, Technical, Billing, Account) that handle domain-specific customer queries with tailored responses.
- **RAG-Based Information Retrieval:** A Retrieval-Augmented Generation pipeline using ChromaDB and HuggingFace embeddings that grounds responses in actual company documents, ensuring accuracy and relevance.
- **SQLite Conversation Memory:** Persistent storage of every customer interaction, enabling the system to recall previous support issues across sessions.
- **Human-in-the-Loop Approval:** A safety mechanism that flags high-risk requests (refunds, cancellations, account closures) for manual human review before processing.
- **Supervisor Validation:** A final quality assurance layer that validates, formats, and professionally presents the response before delivery to the customer.
- **End-to-End Automation:** A unified `app.py` entry point that seamlessly executes the complete workflow from customer input to final response, demonstrating production-ready system integration.

The system successfully handles all five demonstration queries, covering sales inquiries, account management, technical support, billing with human approval, and memory recall — showcasing the full capabilities of the platform.